

第三次实验：使用 MPI 构建多进程倒排索引

2023302121009 刘鼎琨

一、实验源码

1. TODO 1:

```
52 InvertedIndex BuildLocalIndex(const std::vector<Document>& docs) {
53     InvertedIndex index;
54     for (const auto& doc : docs) {
55         std::ifstream fin(doc.path);
56         if (!fin) {
57             continue;
58         }
59
60         std::set<std::string> unique_words;
61         std::string line;
62         while (std::getline(fin, line)) {
63             std::string word;
64             for (unsigned char ch : line) {
65                 if (std::isalpha(ch)) {
66                     word.push_back(static_cast<char>(std::tolower(ch)));
67                 } else if (word.size() >= 2) {
68                     unique_words.insert(word);
69                     word.clear();
70                 } else {
71                     word.clear();
72                 }
73             }
74             if (word.size() >= 2) {
75                 unique_words.insert(word);
76             }
77         }
78
79         for (const auto& word : unique_words) {
80             index[word].insert(doc.id);
81         }
82     }
83     return index;
84 }
```

图 1 TODO 1 代码

2. TODO 2:

```

86 ~ std::string SerializeIndex(const InvertedIndex& index) {
87     std::ostringstream out;
88     for (const auto& [word, docs] : index) {
89         out << word << "\t";
90         bool first = true;
91         for (int doc_id : docs) {
92             if (!first) {
93                 out << ",";
94             }
95             first = false;
96             out << doc_id;
97         }
98         out << "\n";
99     }
100     return out.str();
101 }

```

图 2 TODO 2 代码

3. TODO 3:

```

103 void MergeSerializedIndex(InvertedIndex& global, const std::string& payload) {
104     std::stringstream payload_stream(payload);
105     std::string line;
106     while (std::getline(payload_stream, line)) {
107         if (line.empty()) {
108             continue;
109         }
110
111         std::size_t tab = line.find('\t');
112         if (tab == std::string::npos) {
113             continue;
114         }
115
116         std::string word = line.substr(0, tab);
117         std::string ids = line.substr(tab + 1);
118         std::stringstream id_stream(ids);
119         std::string id_text;
120         while (std::getline(id_stream, id_text, ',')) {
121             if (!id_text.empty()) {
122                 global[word].insert(std::stoi(id_text));
123             }
124         }
125     }
126 }

```

图 3 TODO 3 代码

二、运行结果

由于我安装编译文件的路径不是默认路径,提供的 bat 文件无法找到对应的 MPI 服务,我选择用最基础的指令编译 cpp 文件。

```

~/Desktop $ g++ mpi_inverted_index_starter.cpp -o mpi_inverted_index_starter.exe -I"/extract_msmapi_sdk/msmpisdk_extract/PFiles/Microsoft SDKs/MPI/Include" -L"/extract_msmapi_sdk/msmpisdk_extract/PFiles/Microsoft SDKs/MPI/Lib/x64" -lmsmpi
~/Desktop $ "D:\Program Files\Microsoft MPI\Bin\mpiexec.exe" -n 4 mpi_inverted_index_starter.exe
mpi_size:      4
documents:    297
indexed_docs: 297
unique_terms: 236192
reference_terms: 236192
posting_pairs: 2451950
elapsed_sec:   28.4933
output:       output_global_index.txt
reference:     data/reference_index.tsv
check:        matched
status:       PASS

```

图 4 运行结果截图

三、加速比测试

进程数 (MPI Size)	运行耗时 (sec)	加速比 (Speedup)	效率
1 (基准)	104.18	1.00	100%
2	56.3722	1.848	92.4%
4	28.4933	3.656	91.4%
8	29.7923	3.497	43.7%

通过调整参数形成上表的加速比测试结果，将其绘制为图 5 的折线图：

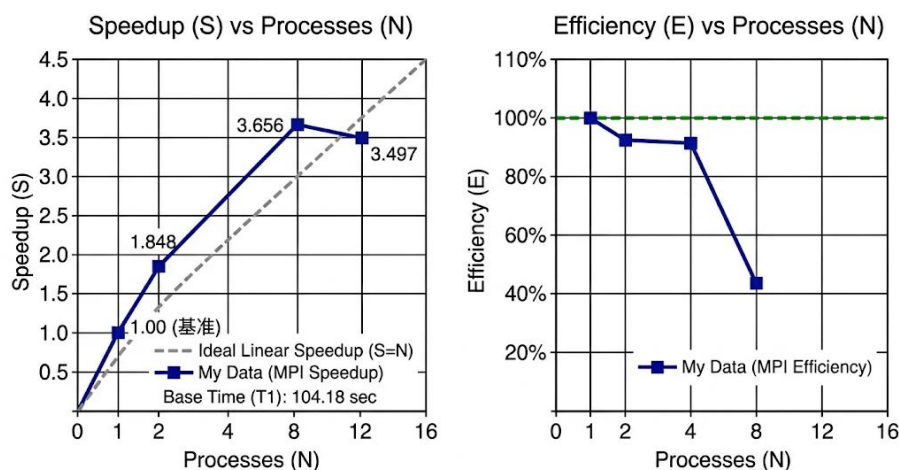


图 5 性能折线图

四、问题分析

Q1: MPI 进程为什么不能直接共享普通变量？

(1) 内存空间独立：MPI 是为分布式内存架构设计的。每个进程在操作系

统中拥有独立的虚拟地址空间。

(2) 硬件物理限制：在真实的集群环境中，不同进程可能运行在不同的物理服务器上，它们的内存条物理上就不互通。MPI 强制开发者使用“消息传递”来交换数据，从而避免了传统多线程中的“竞态条件”和复杂的锁机制。

Q2: 局部索引为什么需要发送给 Rank 0?

每个 Worker 进程只看到了部分文档（由 $i \% \text{size}$ 决定），生成的局部索引是不完整的。实验要求输出一个统一的 `output_global_index.txt`。必须有一个中心节点（Rank 0）来汇总所有局部信息，解决同一个单词出现在不同 Worker 负责的文档中的合并问题。

Q3: 如果进程数增加但不加速，开销在哪里？

(1) I/O 瓶颈：所有进程都在同时读取硬盘上的 `books/` 文件夹，磁盘 IO 达到上限。

(2) 通信开销：序列化后的字符串很大，网络（或本地进程间通信）传输耗时显著。

(3) Rank 0 合并压力：合并逻辑是串行的。Worker 算得再快，如果 Rank 0 解析字符串和插入 `std::map` 的速度跟不上，整体时间就会被拖慢。